SUMMARY: PANDAS-I

SESSION OVERVIEW:

In this session, the students will be able to:

- Understand what Pandas is and its importance; how to install it
- Understand data structures in Pandas
- Understand what a Series is
- Understand the difference between Pandas Series and NumPy Arrays
- Understand the attributes and methods of Series
- Understand what a DataFrame is and how to create one
- Understand the attributes and methods of Dataframe
- Understand the mathematical operations on Series

KEY POINTS AND EXAMPLES:

Understand what Pandas is and its importance

PANDAS (PANel DAta) is a high-level data manipulation tool used for analysing data.

It is very easy to import and export data using Pandas library which has a very rich set of functions.

It is built on packages like NumPy and Matplotlib and gives us a single, convenient place to do most of our data analysis and visualisation work.

Importance of Pandas

- Pandas provides a simple and intuitive syntax for performing data operations, making it accessible even for beginners. The syntax is designed to be consistent and concise, allowing users to perform complex data manipulations with just a few lines of code.
- Built on top of NumPy, Pandas leverages the efficiency of NumPy's array operations. This allows Pandas to handle large datasets efficiently, making it suitable for big data analysis.
- Pandas can handle a wide variety of data formats and sources, including CSV files, Excel spreadsheets, SQL databases, JSON, and more. This versatility makes it a go-to tool for data preprocessing, cleaning, and transformation in many data workflows.
- Data cleaning is a crucial step in any data analysis process, and Pandas provides a range of functions to detect and handle missing or inconsistent data. This helps in preparing the data for analysis and modeling.
- Pandas provides robust tools for data analysis, including statistical functions,

aggregation, and grouping operations. It allows analysts to explore and summarize datasets quickly and effectively.

Installing Pandas

Installing Pandas is very similar to installing NumPy.To install Pandas, we need to type:

```
conda install pandas
```

or

```
pip install pandas
```

Note: Both NumPy and Pandas can be installed only when Python is already installed on that system. The same is true for other libraries of Python.

Importing Pandas

To access Pandas and its functions import it in Python code like this:

```
import pandas as pd
```

Understand data structures in Pandas

A data structure is a collection of data values and operations that can be applied to that data. It enables efficient storage, retrieval and modification to the data.

For example, we have already worked with a data structure ndarray in NumPy.

Pandas offers two primary data structures: Series and DataFrame. These data structures are fundamental for working with data in Pandas, providing powerful tools for data manipulation and analysis.

Understand what a Series is

A Series is a one-dimensional array-like structure in Pandas that can hold a sequence of values of any data type (integers, floats, strings, Python objects, etc.). Each value in a Series is associated with a unique label, known as the index. By default, the index is a sequence of integers starting from zero, but it can be customized to use any set of labels.

Each element in a Series is of the same data type, though Pandas can handle mixed data

types.
Each element in a Series is associated with an index label, which can be numeric or non-numeric.
A Series is mutable, meaning that you can change its values.

A Series can be thought of as a single column in a spreadsheet or a single column in a database table.

Example of a Series containing names of students is given below:

```
Index     Value
0         Arnab
1         Samridhi
2         Ramit
3         Divyam
4         Kritika
```

Understand the difference between Pandas Series and NumPy Arrays

Pandas Series	NumPy Arrays
In series we can define our own labeled index to access elements of an array. These can be numbers or letters.	NumPy arrays are accessed by their integer position using numbers only
If two series are not aligned, NaN or missing values are generated.	There is no concept of NaN values and if there are no matching values in arrays, alignment fails.
Series require more memory.	NumPy occupies lesser memory.

Creation of Series

There are different ways in which a Series can be created in Pandas. To create or use Series, we first need to import the Pandas library.

A) Creation of Series from Scalar Values

A "scalar value" typically refers to a single value. A Series can be created using scalar values as shown in the example below:

```
import pandas as pd

# Creating a Series from a scalar value (with custom index)
s_scalar = pd.Series(10, index=['a', 'b', 'c', 'd', 'e'])
print(s_scalar)
```

The output will be:

```
a    10
b    10
c    10
d    10
e    10
dtype: int64
```

Here, 10 is the scalar value repeated for each index label ('a', 'b', 'c', 'd', 'e'). The default data type inferred by Pandas for this scalar value is int64.

Note: If you create a Pandas Series without specifying an index parameter, Pandas will automatically generate a default numeric index starting from 0.

```
import pandas as pd

# Creating a Series from a scalar value (without specifying index)
s_scalar = pd.Series(10)
print(s_scalar)
```

The output will be:

```
0    10
dtype: int64
```

In this example:

- The scalar value 10 is repeated once.
- The Series has a default numeric index starting from 0 (0 in this case).

This is useful when you want to initialize a Series with a default value or when you plan to fill the Series with data later.

B) Creating a Series from a List

We can create a Series directly from a Python list. By default, it will have a numeric index starting from 0.

```
import pandas as pd

# Creating a Series from a list
data = [10, 20, 30, 40, 50]
s = pd.Series(data)
```

```
print(s)
```

The output will be:

```
0    10
1    20
2    30
3    40
4    50
dtype: int64
```

We can specify a custom index by passing an index list along with the data.

```
# Creating a Series from a list with a custom index
s_custom_index = pd.Series(data, index=['a', 'b', 'c', 'd', 'e'])
print(s_custom_index)
```

The output will be:

```
a    10
b    20
c    30
d    40
e    50
dtype: int64
```

If the number of index labels does not match the number of data elements, Pandas will raise a ValueError.

```
import pandas as pd

# Attempting to create a Series with mismatched data and index
data = [10, 20, 30, 40, 50]
s_custom_index = pd.Series(data, index=['a', 'b', 'c', 'd'])  #
Missing 'e'
print(s_custom_index)
```

The output will be:

```
ValueError: Length of values (5) does not match length of index
(4)
```

Handling Mismatched Data and Index: If you encounter a situation where the number of data elements and index labels may not match, you have a few options:

1. Ensure that the number of index labels matches the number of data elements.
2. Let Pandas assign a default numeric index if specific index labels are not necessary or if they are not available.

C) Creating a Series from a NumPy Array

We can also create a Series from a NumPy array, which is useful for numerical operations.

```
import pandas as pd
import numpy as np

# Creating a NumPy array
data_array = np.array([10, 20, 30, 40, 50])

# Creating a Series from a NumPy array with custom index
s_custom_index = pd.Series(data_array, index=['a', 'b', 'c', 'd',
'e'])
print(s_custom_index)
```

The output will be:

```
a    10
b    20
c    30
d    40
e    50
dtype: int32
```

D) Creating a Series from a Dictionary

We can create a Series from a Python dictionary. The keys become the index of the Series.

```
import pandas as pd
```

```
# Creating a Series from a dictionary
data_dict = {'a': 10, 'b': 20, 'c': 30, 'd': 40, 'e': 50}
s_dict = pd.Series(data_dict)
print(s_dict)
```

The output will be:

```
a    10
b    20
c    30
d    40
e    50
dtype: int64
```

When creating a Pandas Series from a dictionary, you typically do not need to use the index parameter explicitly. The keys of the dictionary automatically become the index labels of the Series. However, if you want to ensure a specific order of index labels or if you want to select a subset of keys from the dictionary, you can use the index parameter.

```
# Specifying index labels explicitly
s_dict_custom_index = pd.Series(data_dict, index=['b', 'c', 'a'])
print(s_dict_custom_index)
```

The output will be:

```
b    20
c    30
a    10
dtype: int64
```

Accessing Elements of a Series

There are two common ways for accessing the elements of a series: Indexing and Slicing.

A) **Indexing:** Indexing in Series is similar to that for NumPy arrays, and is used to access elements in a series. Indexes are of two types: positional index and labelled index. Positional index takes an integer value that corresponds to its position in the series starting from 0, whereas labelled index takes any user-defined label as index.

Following example shows usage of the positional index for accessing a value from a Series.

```
import pandas as pd

# Creating a Series
data = [10, 20, 30, 40, 50]
s = pd.Series(data)
# Accessing a single element by positional index
print(s[0])
```

The output will be:

```
10
```

When labels are specified, we can use labels as indices while selecting values from a Series, as shown below.

```
import pandas as pd

# Creating a Series
data = [10, 20, 30, 40, 50]
s = pd.Series(data, index=['a', 'b', 'c', 'd', 'e'])

# Accessing a single element by index label
print(s['a'])  # Output: 10

# Accessing multiple elements by index labels (using a list)
print(s[['a', 'c', 'e']])
```

The output will be:

```
10
a    10
c    30
e    50
dtype: int64
```

We can access the values from a dictionary using keys.

```
import pandas as pd

# Creating a Series from a dictionary
data = {'Alice': 85, 'Bob': 90, 'Charlie': 78}
```

```
series = pd.Series(data)

# Accessing values using keys (index labels)
print(series['Alice'])
print(series['Bob'])
print(series['Charlie'])
```

The output will be:

```
85
90
78
```

B) **Slicing:** Sometimes, we may need to extract a part of a series. This can be done through slicing. This is similar to slicing used with NumPy arrays. We can define which part of the series is to be sliced by specifying the start and end parameters [start :end] with the series name.

```
import pandas as pd

# Creating a Series
data = [10, 20, 30, 40, 50]
s = pd.Series(data)
# Slicing using positional index (inclusive of 1 to 3)
slice_by_positions = s[1:4]
print(slice_by_positions)
```

The output will be:

```
1    20
2    30
3    40
dtype: int64
```

```
import pandas as pd

# Creating a Series
data = [10, 20, 30, 40, 50]
s = pd.Series(data, index=['a', 'b', 'c', 'd', 'e'])
```

```
# Slicing using index labels (inclusive of 'b' to 'd')
slice_by_labels = s['b':'d']
print(slice_by_labels)
```

The output will be:

```
b    20
c    30
d    40
dtype: int64
```

Note: Slicing in Pandas Series behaves similarly to slicing in Python lists or NumPy arrays. When slicing by positional index, the end position is exclusive, whereas slicing by index labels is inclusive of both start and end labels.

(Comment for instructor: Just provide basic information on the attributes, methods, and mathematical operations of Series.)

Understand the attributes of Series

We can access certain properties called attributes of a series by using that property with the series name.

Let's create a Pandas Series, where countries are the index and their capitals are the values.

```
import pandas as pd

# Data for the Series
data = ['NewDelhi', 'WashingtonDC', 'London', 'Paris']
index = ['India', 'USA', 'UK', 'France']

# Creating the Series
seriesCapCntry = pd.Series(data, index=index)

print(seriesCapCntry)
```

The output will be:

```
India         NewDelhi
USA       WashingtonDC
UK              London
France           Paris
```

```
dtype: object
```

Let's perform all the attributes on the seriesCapCntry Pandas Series that we created:

1. **index:** Returns the index (labels) of the Series. (in DF too)

```
print("Index:")
print(seriesCapCntry.index)

#Output
Index:
Index(['India', 'USA', 'UK', 'France'], dtype='object')
```

2. **values:** Returns the values of the Series. (in DF too)

```
print("Values:")
print(seriesCapCntry.values)

#Output
Values:
['NewDelhi' 'WashingtonDC' 'London' 'Paris']
```

3. **name:** Returns or sets the name of the Series.

```
# Setting the name of the Series
Serie
sCapCntry.name = 'Capitals'
print("Name:")
print(seriesCapCntry.name)

#Output
Name:
Capitals
```

4. **dtype:** Returns the data type of the values in the Series. (dtypes in DF)

```
print("Dtype:")
print(seriesCapCntry.dtype)

#Output
Dtype:
object
```

5. **shape:** Returns the shape of the Series (number of elements).(in DF too)

```
print("Shape:")
print(seriesCapCntry.shape)

#Output
Shape:
(4,)
```

6. **size:** Returns the number of elements in the Series. (in DF too)

```
print("Size:")
print(seriesCapCntry.size)

#Output
Size:
4
```

7. **empty:** Returns True if the Series is empty. (in DF too)

```
print("Empty:")
print(seriesCapCntry.empty)

# Output
Empty:
False
```

8. **hasnans:** Returns True if the Series contains any NaN values. (in DF too)

```
print("Has NaNs:")
print(seriesCapCntry.hasnans)

# Output
Has NaNs:
False
```

9. **is_unique:** Returns True if all the elements in the Series are unique.

```
print("Is Unique:")
print(seriesCapCntry.is_unique)

#Output
Is Unique:
True
```

10. **ndim:** Returns the number of dimensions of the Series (always 1 for a Series). (in DF

too)

```
print("Ndim:")
print(seriesCapCntry.ndim)

#Output
Ndim:
1
```

Attributes for Pandas Series but not for DataFrame:

1. .name
2. .is_unique
3. .is_monotonic
4. .itemsize
5. .nbytes

Undertsand the methods of Series

Pandas Series comes with a variety of methods that allow for efficient data manipulation and analysis.

Let's create a new Pandas Series with student names as the index and their corresponding marks as the values. Then, we'll perform all the methods on this new Series.

```
import pandas as pd

# Data for the Series
marks = [85, 90, 78, 92, 88]
students = ['Alice', 'Bob', 'Charlie', 'David', 'Eva']

# Creating the Series
seriesStudents = pd.Series(marks, index=students)

print("Series:")
print(seriesStudents)
```

The output will be:

```
Series:
Alice      85
Bob        90
Charlie    78
David      92
```

```
Eva        88
dtype: int64
```

1. **head()**: Returns the first n elements of the Series. If the value for n is not passed, then by default n takes 5 and the first five members are displayed.

```
print("Head:")
print(seriesStudents.head(2))

#Output
Head:
Alice    85
Bob      90
dtype: int64
```

2. **tail():** Returns the last n elements of the Series. . If the value for n is not passed, then by default n takes 5 and the last five members are displayed.

```
print("Tail:")
print(seriesStudents.tail(2))

#Output
Tail:
David    92
Eva      88
dtype: int64
```

3. **describe():** Generates descriptive statistics.

```
print("Describe:")
print(seriesStudents.describe())

#Output
Describe:
count     5.000000
mean     86.600000
std       5.458938
min      78.000000
25%      85.000000
50%      88.000000
75%      90.000000
max      92.000000
dtype: float64
```

4. **value_counts():** Returns a Series containing counts of unique values.

```
print("Value Counts:")
print(seriesStudents.value_counts())

#Output
Value Counts:
85    1
90    1
78    1
92    1
88    1
Name: count, dtype: int64
```

5. **sort_values():** Sorts the Series by its values. it sorts the values in ascending order by default. However, we can sort the values in descending order by setting the ascending parameter to False (ascending=False).

```
print("Sort Values:")
print(seriesStudents.sort_values())

#Output
Sort Values:
Charlie    78
Alice      85
Eva        88
Bob        90
David      92
dtype: int64
```

6. **sort_index():** Sorts the Series by its index in ascending order by default. If you want to sort the index in descending order, you can use the ascending=False parameter:

```
print("Sort Index:")
print(seriesStudents.sort_index())

#Output
Sort Index:
Alice      85
Bob        90
Charlie    78
```

```
David      92
Eva        88
dtype: int64
```

7. **drop():** Removes the specified index labels from the Series.

```
print("Drop (Charlie):")
print(seriesStudents.drop('Charlie'))

#Output
Drop (Charlie):
Alice    85
Bob      90
David    92
Eva      88
dtype: int64
```

8. **replace():** Replaces specified values in the Series.

```
print("Replace (85 with 86):")
print(seriesStudents.replace(85, 86))

#Output
Replace (85 with 86):
Alice      86
Bob        90
Charlie    78
David      92
Eva        88
dtype: int64
```

9. **isnull():** Detects missing values. sum() can be used with isnull() to count the number of missing values. Since True is interpreted as 1 and False as 0 when summing boolean values, summing the result of isnull() gives the count of missing values.

```
print("Is Null:")
print(seriesStudents.isnull())

#Output
Is Null:
Alice      False
Bob        False
Charlie    False
```

```
David       False
Eva         False
dtype: bool
```

```
# Counting missing values using sum()
missing_count = seriesStudents.isnull().sum()
print("Number of missing values:")
print(missing_count)

#Output
Number of missing values:
0
```

10. **notnull():** Detects non-missing values.

```
print("Not Null:")
print(seriesStudents.notnull())

#Output
Not Null:
Alice       True
Bob         True
Charlie     True
David       True
Eva         True
dtype: bool
```

```
# Counting non-missing values using sum()
not_null_count = seriesStudents.notnull().sum()
print("Number of non-missing values:")
print(not_null_count)

# Output
Number of non-missing values:
5
```

Note: By default, the methods we applied to the Series create and return a new Series with the modifications, leaving the original Series unchanged. To modify the original Series, you would need to either reassign the result to the original Series or use the inplace=True parameter if the method supports it.

```
# Dropping an element without inplace
dropped_series = seriesStudents.drop('Charlie')
print("Dropped Series (without inplace):")
print(dropped_series)

# Original Series remains unchanged
print("\nOriginal Series (unchanged):")
print(seriesStudents)
```

The Output will be:

```
Dropped Series (without inplace):
Alice    85
Bob      90
David    92
Eva      88
dtype: int64

Original Series (unchanged):
Alice      85
Bob        90
Charlie    78
David      92
Eva        88
dtype: int64
```

```
# Dropping an element with inplace
seriesStudents.drop('Charlie', inplace=True)
print("Original Series after drop with inplace=True:")
print(seriesStudents)
```

The output will be:

```
Original Series after drop with inplace=True:
Alice    85
Bob      90
David    92
Eva      88
dtype: int64
```

- Without inplace:
    - The drop method returns a new Series.
    - The original Series remains unchanged.
- With inplace=True:
    - The drop method modifies the original Series directly.
    - No new Series is returned.

Understand the mathematical operations on Series

If we perform basic mathematical operations like addition, subtraction, multiplication, division, etc., on two NumPy arrays, the operation is done on each corresponding pair of elements. Similarly, we can perform mathematical operations on two series in Pandas.

Pandas Series supports various mathematical operations that can be applied element-wise. These operations can be used to perform arithmetic calculations, comparisons, and other mathematical manipulations.

While performing mathematical operations on series, index matching is implemented and all missing values are filled in with NaN by default. means that when you perform mathematical operations (such as addition, subtraction, multiplication, etc.) on two Pandas Series, Pandas aligns the Series based on their indices. If an index is present in one Series but not in the other, the resulting value for that index will be set to `NaN` (Not a Number). This default behavior ensures that the operation can handle mismatched indices without causing errors, but it also highlights the presence of missing data.

Consider the following series: seriesA and seriesB for understanding mathematical operations on series in Pandas.

```
import pandas as pd
seriesA = pd.Series([1,2,3,4,5], index =['a', 'b', 'c', 'd', 'e'])
print('seriesA:')
print(seriesA)
seriesB = pd.Series([10,20,-10,-50,100],index = ['z', 'y', 'a',
'c', 'e'])
print('seriesB')
print(seriesB)
```

The output will be:

```
seriesA:
```

```
a    1
b    2
c    3
d    4
e    5
dtype: int64
seriesB
z     10
y     20
a    -10
c    -50
e    100
dtype: int64
```

A) Addition of two Series

```
# Performing addition
result_addition = seriesA + seriesB
print("Addition of seriesA and seriesB:")
print(result_addition)
```

The output will be:

```
Addition of seriesA and seriesB:
a     -9.0
b      NaN
c    -47.0
d      NaN
e    105.0
y      NaN
z      NaN
dtype: float64
```

Table shows the detailed values that were matched while performing the addition.

index	value from series A	value from series B	SeriesA + seriesB
a	1	-10	-9.0
b	2		NaN
c	3	-50	-47.0

d	4		NaN
e	5	100	105.0
y		20	NaN
z		10	NaN

Handling NaN Values

Note: the same methods for handling NaN values (filling with a specific value or dropping them) can be applied to other mathematical operations on Series, such as addition, subtraction, multiplication, and division.

1. Filling NaN Values: You can handle NaN values by filling them with a specific value using the fillna() method:

```
result_addition_filled = result_addition.fillna(0)
print("Addition result after filling NaN with 0:")
print(result_addition_filled)
```

The output will be:

```
Addition result after filling NaN with 0:
a     -9.0
b      0.0
c    -47.0
d      0.0
e    105.0
y      0.0
z      0.0
dtype: float64
```

2. Dropping NaN Values: You can also remove NaN values using the dropna() method:

```
result_addition_dropped = result_addition.dropna()
print("Addition result after dropping NaN values:")
print(result_addition_dropped)
```

The output will be:

```
Addition result after dropping NaN values:
a     -9.0
```

```
c    -47.0
e    105.0
dtype: float64
```

B) Subtraction of two Series

```
# Performing subtraction
result_subtraction = seriesA - seriesB
print("Subtraction of seriesB from seriesA:")
print(result_subtraction)
```

The output will be:

```
Subtraction of seriesB from seriesA:
a    11.0
b     NaN
c    53.0
d     NaN
e   -95.0
y     NaN
z     NaN
dtype: float64
```

C) Multiplication of two Series

```
result_multiplication = seriesA * seriesB
print("Multiplication of seriesA and seriesB:")
print(result_multiplication)
```

The output will be:

```
Multiplication of seriesA and seriesB:
a    -10.0
b      NaN
c   -150.0
d      NaN
e    500.0
y      NaN
z      NaN
dtype: float64
```

D) Division of two Series

```
result_division = seriesA / seriesB
print("Division of seriesA by seriesB:")
print(result_division)
```

The output will be:

```
Division of seriesA by seriesB:
a   -0.10
b     NaN
c   -0.06
d     NaN
e    0.05
y     NaN
z     NaN
dtype: float64
```

Understand what a DataFrame is

(Comment for instructor: Make sure to tell the students that in a Jupyter Notebook, when you simply write the name of a variable or an expression that results in a DataFrame, it automatically displays the DataFrame without needing an explicit print() function. In a standard Python script, the output is plain text without the rich formatting provided by Jupyter Notebook.)

Sometimes we need to work on multiple columns at a time, i.e., we need to process the tabular data. For example, the result of a class, items in a restaurant's menu, reservation chart of a train, etc.

Pandas store such tabular data using a DataFrame. A DataFrame is a two-dimensional labelled data structure like a table of MySQL. It contains rows and columns, and therefore has both a row and column index. Each column can have a different type of value such as numeric, string, boolean, etc., as in tables of a database.

Example of a DataFrame that includes information about students, their grades in different subjects, and their overall status.

```
   Student  Math  Science  English  Passed
0    Alice    85       92       88    True
```

```
1      Bob    90       88       79    True
2  Charlie    78       84       85   False
3    David    88       95       82    True
```

Creation of DataFrame

There are different ways in which a DataFrame can be created in Pandas. To create or use DataFrame, we first need to import the Pandas library.

1. **Creation of an empty DataFrame:** We can create an empty DataFrame.

```
import pandas as pd

# Creating an empty DataFrame
empty_df = pd.DataFrame()

# Displaying the empty DataFrame
print(empty_df)
```

The output will be:

```
Empty DataFrame
Columns: []
Index: []
```

2. **Creating a DataFrame from a NumPy Array**: We can create a DataFrame from a NumPy array by specifying the column names.

```
import numpy as np
import pandas as pd

# Example data
data = np.array([
    [25, 'New York'],
    [30, 'Los Angeles'],
    [35, 'Chicago'],
    [40, 'Houston']
])

# Creating DataFrame
df = pd.DataFrame(data, columns=['Age', 'City'])
```

```
print(df)
```

The output will be:

```
  Age         City
0  25     New York
1  30  Los Angeles
2  35      Chicago
3  40      Houston
```

3. **Creating a DataFrame from a List of Dictionaries:** Each dictionary in the list represents a row of data. This method is useful when your data is naturally structured as dictionaries.

```
import pandas as pd

data = [
    {'Name': 'Alice', 'Age': 25, 'City': 'New York'},
    {'Name': 'Bob', 'Age': 30, 'City': 'Los Angeles'},
    {'Name': 'Charlie', 'Age': 35, 'City': 'Chicago'},
    {'Name': 'David', 'Age': 40, 'City': 'Houston'}
]

df = pd.DataFrame(data)
print(df)
```

The output will be:

```
Name  Age         City
0    Alice   25     New York
1      Bob   30  Los Angeles
2  Charlie   35      Chicago
3    David   40      Houston
```

We can create a DataFrame by importing data from CSV files where values are separated by commas. Similarly, we can also store or export data in a DataFrame as a .csv file.

Understand how to create a DataFrame from a Excel/CSV file

We have three datasets given below:
Customers
Products
Purchase

To create a DataFrame from a Excel file, we use the pd.read_excel() function provided by Pandas

```
import pandas as pd
Customers = pd.read_excel("customers.xlsx")
Products = pd.read_excel("products.xlsx")
Purchase = pd.read_excel("purchases.xlsx")
```

Better way of achieving the above

```
import pandas as pd

try:
    Customers = pd.read_excel("path/to/customers.xlsx")
    Products = pd.read_excel("path/to/products.xlsx")
    Purchase = pd.read_excel("path/to/purchases.xlsx")
except FileNotFoundError as e:
    print(f"File not found: {e}")
```

Note: for Excel files with multiple sheets, we can also specify the sheet name by passing the sheet_name parameter to the pd.read_excel() function.

```
# Read a specific sheet by name
df_specific_sheet = pd.read_excel('customers.xlsx',
sheet_name='customers')
```

To create a DataFrame from a CSV file, we use the pd.read_csv() function provided by Pandas

```
import pandas as pd
Customers = pd.read_csv("customers.csv")
Products = pd.read_csv("products.csv")
Purchase = pd.read_csv("purchases.csv")
```

Note: Provide the correct file paths for the files located in your system.

To print the type of a DataFrame in Python using the pandas library, we can use the built-in type() function.

```
print(type(Customers))
```

The output will be:

```
<class 'pandas.core.frame.DataFrame'>
```

Accessing DataFrames Element through Indexing/Slicing

1. **Accessing a Single Column:** To access a single column in a DataFrame, We can use the column name as a key.

Syntax:

```
column_data = DataFrame['column_name']
```

```
# Accessing the 'first_name' column
customer_names = Customers['first_name']
print("Customer Names:")
print(customer_names)
```

In this example:

- The Customers DataFrame is loaded from an Excel.
- The Customers['first_name'] statement accesses the 'first_name' column.
- The print(customer_names) statement prints the accessed column values.

The output will be:

```
Customer Names:
0       Romain
1       Cosimo
2        Bambi
3       Roarke
4      Mikaela
        ...
995    Merrili
996    Winonah
997      Tobit
998     Issiah
999     Elmore
Name: first_name, Length: 1000, dtype: object
```

Using the .loc Accessor: .loc is primarily used for label-based indexing, which directly applies to columns as they are accessed by their names (labels).

Syntax:

```
column_data = DataFrame.loc[:, 'column_name']
```

```
# Using .loc to access specific columns by label
first_column_loc = Customers.loc[:, 'id']  # Accessing 'id' column
by label
second_column_loc = Customers.loc[:, 'first_name']  # Accessing
'first_name' column by label

print("First Column using .loc:")
print(first_column_loc)

print("\nSecond Column using .loc:")
print(second_column_loc)
```

The output will be:

```
First Column using .loc:
0         1
1         2
2         3
3         4
4         5
       ...
995     996
996     997
997     998
998     999
999    1000
Name: id, Length: 1000, dtype: int64

Second Column using .loc:
0       Romain
1       Cosimo
2        Bambi
3       Roarke
4      Mikaela
        ...
995    Merrili
996    Winonah
997      Tobit
998     Issiah
999     Elmore
Name: first_name, Length: 1000, dtype: object
```

Using the .iloc Accessor: The .iloc accessor is used for positional indexing. You specify the integer position of the column you want to access.

Syntax:

```
columns_data = DataFrame.iloc[:, column_index]
```

```
# Using .iloc to access specific columns by integer position
first_column_iloc = Customers.iloc[:, 0]  # Accessing first column
by position
second_column_iloc = Customers.iloc[:, 1]  # Accessing second
column by position

print("First Column using .iloc:")
print(first_column_iloc)

print("\nSecond Column  using .iloc:")
print(second_column_iloc)
```

The output will be:

```
First Column using .iloc:
0         1
1         2
2         3
3         4
4         5
       ...
995     996
996     997
997     998
998     999
999    1000
Name: id, Length: 1000, dtype: int64

Second Column  using .iloc:
0       Romain
1       Cosimo
2        Bambi
3       Roarke
4      Mikaela
        ...
995    Merrili
996    Winonah
997      Tobit
998     Issiah
999     Elmore
Name: first_name, Length: 1000, dtype: object
```

Both .loc and .iloc can be used to access columns in a DataFrame, but they use different methods:

- Use .loc when you know the column labels.
- Use .iloc when you know the column positions.

In this example:

- The first_column_loc and first_column_iloc give the same result because .loc[:, 'id'] accesses the 'id' column by its label, and .iloc[:, 0] accesses the first column by its position.
- The second_column_loc and second_column_iloc give the same result because .loc[:, 'first_name'] accesses the 'first_name' column by its label, and .iloc[:, 1] accesses the second column by its position.
- The colon (:) within the .loc method and .iloc method is used to indicate that you want to select all rows.

2. **Accessing Multiple Columns:** To access multiple columns in a DataFrame, you can use the column names as keys.

Syntax:

```
columns_data = DataFrame[['column_name1', 'column_name2',
```

```
'column_name3']]
```

```
# Accessing multiple columns using column names as keys
customer_info = Customers[['first_name','last_name', 'city']]
print("Customer Info:")
print(customer_info)
```

The output will be:

```
Customer Info:
    first_name    last_name          city
0       Romain    Southcott     San Diego
1       Cosimo    Molyneaux       El Paso
2        Bambi      Westrip   San Antonio
3       Roarke   Pankettman       Memphis
4      Mikaela     Althorpe           NaN
..         ...          ...           ...
995    Merrili        Alman       Reading
996    Winonah       Heckle  Indianapolis
997      Tobit         Birt     Waterbury
998     Issiah  Standbrooke      Savannah
999     Elmore       Malpas       Atlanta

[1000 rows x 3 columns]
```

Using the .loc Accessor

Syntax:

```
columns_data = DataFrame.loc[:, ['column_name1', 'column_name2',
'column_name3']]
```

```
# Accessing multiple columns using .loc
customer_info_loc = Customers.loc[:, ['first_name', 'city']]
print("Customer Info using .loc (first_name and City):")
print(customer_info_loc)
```

The output will be:

```
Customer Info using .loc (first_name and City):
    first_name          city
0       Romain     San Diego
1       Cosimo       El Paso
2        Bambi   San Antonio
3       Roarke       Memphis
4      Mikaela           NaN
..         ...           ...
995    Merrili       Reading
996    Winonah  Indianapolis
997      Tobit     Waterbury
998     Issiah      Savannah
999     Elmore       Atlanta

[1000 rows x 2 columns]
```

Using the .iloc Accessor

Syntax:

```
columns_data = DataFrame.iloc[:, [column_index1, column_index2,
...]]
```

```
# Accessing multiple columns using .iloc
customer_info_iloc = Customers.iloc[:, [0, 2]]
print("Customer Info using .iloc (first_name and City):")
print(customer_info_iloc)
```

The output will be:

```
Customer Info using .iloc (first_name and City):
       id          city
0       1     San Diego
1       2       El Paso
2       3   San Antonio
3       4       Memphis
4       5           NaN
..    ...           ...
995   996       Reading
996   997  Indianapolis
997   998     Waterbury
998   999      Savannah
999  1000       Atlanta

[1000 rows x 2 columns]
```

3. **Accessing a Single Row:** To access a single row in a DataFrame, we can use the .loc or .iloc accessors.

A. **Using the .loc Accessor:** The .loc accessor is used for label-based indexing. You specify the index label of the row you want to access.

Syntax:

```
row_data = DataFrame.loc[index_label]
```

```
# Accessing a single row using .loc (label-based indexing)
customer_row_loc = Customers.loc[2]
print("Customer Row (index label 2) using .loc:")
print(customer_row_loc)
```

The output will be:

```
Customer Row (index label 2) using .loc:
id                                    3
first_name                        Bambi
last_name                       Westrip
email             bwestrip2@symantec.com
gender                           Female
street_num                       4057.0
street_name                    Arkansas
street_suffix                    Circle
city                        San Antonio
state                             Texas
postcode                        78220.0
Name: 2, dtype: object
```

B. **Using the .iloc Accessor:** The .iloc accessor is used for positional indexing. You specify the integer position of the row you want to access.

Syntax:

```
row_data = DataFrame.iloc[row_position]
```

```
# Accessing a single row using .iloc (positional indexing)
customer_row_iloc = Customers.iloc[2]
print("Customer Row (position 2) using .iloc:")
print(customer_row_iloc)
```

The output will be:

```
Customer Row (position 2) using .iloc:
id                                   3
first_name                       Bambi
last_name                      Westrip
email            bwestrip2@symantec.com
gender                          Female
street_num                      4057.0
street_name                   Arkansas
street_suffix                   Circle
city                       San Antonio
state                            Texas
postcode                       78220.0
Name: 2, dtype: object
```

Key Differences

.loc uses the index labels. If the index labels are integers and correspond to their positions, it appears the same as .iloc, but their usage intent is different.

.iloc uses the integer position (starting from 0). It doesn't care about the index labels and always refers to the positional location.

4. **Boolean Indexing:** Boolean indexing in pandas allows you to filter a DataFrame based on conditions defined by boolean operations. It involves selecting rows that satisfy a certain condition or set of conditions. Here's how boolean indexing works and its syntax:

Syntax:

```
DataFrame[boolean_condition]
```

```
# Boolean indexing to filter customers who are Male
male_customers = Customers[Customers['gender'] == 'Male']

print("Male Customers:")
male_customers
```

The output will be:

```
Male Customers:
```

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode
0	1	Romain	Southcott	rsouthcott0@clickbank.net	Male	1.0	Trailsway	Road	San Diego	California	92127.0
1	2	Cosimo	Molyneaux	cmolyneaux1@wiley.com	Male	NaN	NaN	NaN	El Paso	Texas	NaN
3	4	Roarke	Pankettman	rpankettman3@wiley.com	Male	74.0	Debs	Point	Memphis	Tennessee	38150.0
10	11	Natale	Martina	nmartinaa@wordpress.com	Male	221.0	Sauthoff	Pass	Bradenton	Florida	34205.0
12	13	Mikol	MacWhan	mmacwhanc@patch.com	Male	9.0	Eagle Crest	Center	Bronx	New York	10469.0
...	...	...	...	...	...	...	...	...	...	...	...
984	985	Tailor	Sealeaf	tsealeafrc@irs.gov	Male	NaN	Summer Ridge	Pass	Beaufort	South Carolina	29905.0
986	987	Eric	Mountford	emountfordre@blogspot.com	Male	NaN	6th	Court	Las Vegas	Nevada	89125.0
997	998	Tobit	Birt	NaN	Male	2.0	Basil	Road	Waterbury	Connecticut	6721.0
998	999	Issiah	Standbrooke	istandbrookerq@yellowpages.com	Male	NaN	Kenwood	Drive	Savannah	Georgia	31405.0
999	1000	Elmore	Malpas	NaN	Male	205.0	Farwell	Park	Atlanta	Georgia	30386.0

485 rows × 11 columns

In this example:

- male_customers = Customers[Customers['gender'] == 'Male'] This line filters the Customers DataFrame to select rows where the 'gender' column is equal to 'Male'.
- By specifying the condition Customers['gender'] == 'Male', you selectively extract rows from the Customers DataFrame where the gender is 'Male'.

```
# Filter products where the company is 'Dynazzy'
filtered_products = Products[Products['company'] == 'Dynazzy']

print("Filtered Products:")
print(filtered_products)
```

The output will be:

```
Filtered Products:
   Unnamed: 0  id                     product    cost  company
1           1   2  Nori Sea Weed - Gold Label  $85.74  Dynazzy
```

We can combine conditions using logical operators like & (and), | (or), and ~ (not) to create complex filters.

```
# Filter rows where first name is 'Tobit' or 'Natale'
filtered_customers = Customers[(Customers['first_name'] ==
'Tobit') | (Customers['first_name'] == 'Natale')]
filtered_customers
```

The output will be:

	id	first_name	last_name	email	gender	street_num	street_name	street_suffix	city	state	postcode
10	11	Natale	Martina	nmartinaa@wordpress.com	Male	221.0	Sauthoff	Pass	Bradenton	Florida	34205.0
997	998	Tobit	Birt	NaN	Male	2.0	Basil	Road	Waterbury	Connecticut	6721.0

filtered_customers: This variable holds the resulting DataFrame after applying the filter. It contains only the rows from Customers where first_name is either 'Tobit' or 'Natale'.

Accessing DataFrames Element through Slicing

To access elements in a DataFrame through slicing, you can use both .loc and .iloc methods in pandas.

```
# Slicing rows and columns using .loc
# Example: Slice rows 2 to 4 (inclusive) and columns 'product_num'
and 'paid'
sliced_loc = Purchase.loc[2:4, ['product_num', 'paid']]

print("Using .loc for slicing:")
print(sliced_loc)
```

The output will be:

```
Using .loc for slicing:
   product_num     paid
2            9  $510.17
3           19   $68.49
4            8  $759.42
```

In this example:

- Purchase.loc[2:4, ['product_num', 'paid']]: This slices rows from index 2 to 4 (inclusive) and selects columns 'product' and 'cost' using label-based indexing with .loc.

```
# Example: Print entire dataset between row indices 2 and 4 using
.loc
sliced_loc = Purchase.loc[2:4, :]
print("Using .loc for slicing:")
print(sliced_loc)
```

The output will be:

```
Using .loc for slicing:
   Unnamed: 0  id           purch_date  customer_num  product_num  amount  \
2           2   3  2019-01-03 00:00:00           955            9      17
3           3   4  2019-01-03 00:00:00           577           19       3
4           4   5  2019-01-03 00:00:00           429            8      18

      paid
2  $510.17
3   $68.49
4  $759.42
```

- Purchase.loc[2:4, :] specifies slicing rows from index 2 to index 4 (inclusive).
- The : in the second position (:) indicates that you want to select all columns in the DataFrame for the specified rows.

```
# Slicing rows and columns using .iloc
# Example: Slice rows 1 to 4 (exclusive) and columns 1 to 3
(exclusive)
sliced_iloc = Purchase.iloc[1:4, 1:3]

print("\nUsing .iloc for slicing:")
print(sliced_iloc)
```

The output will be:

```
Using .iloc for slicing:
   id           purch_date
1   2  2019-01-03 00:00:00
2   3  2019-01-03 00:00:00
3   4  2019-01-03 00:00:00
```

In this example:

- This slices rows from index 1 to 4 (exclusive) and columns from index 1 to 3 (exclusive) using positional indexing with .iloc.

.loc uses inclusive slicing for both rows and columns. This means the endpoint of the slice is included in the result.
.iloc uses exclusive slicing for both rows and columns. This means the endpoint of the slice is not included in the result.